
Reproducible projects in R:

practical workshop, part 2

Dr Marina Vabistsevit

Research Fellow in Clinical and Biomedical Sciences
UK Reproducibility Network (UKRN) Local Network Lead for Exeter

What this workshop is about

Part 1: (last week)

Project-oriented workflow:

- Organising your projects with .Rproj
- Using renv to manage packages in your Rstudio environment
- Reproducibility habits and efficiency tips

Part 2: (today!)

Using Git to track changes in your R project



Icon for R



Icon for RStudio



software
company who
develop
Rstudio

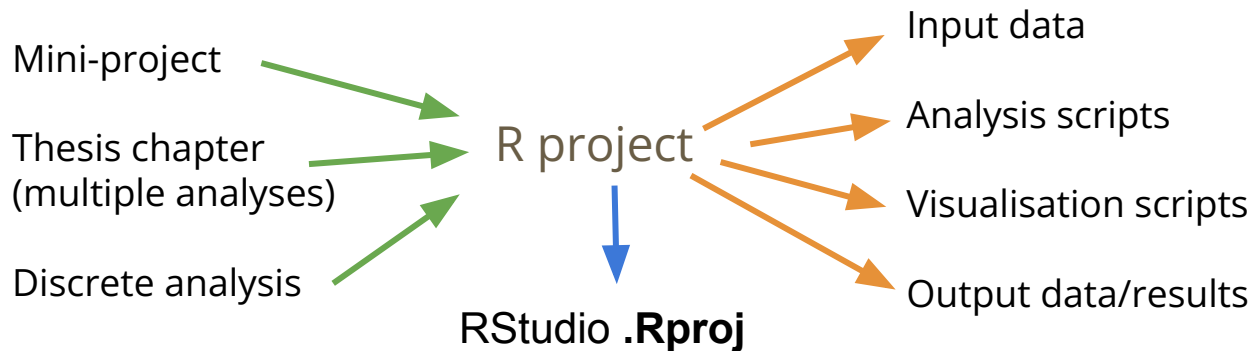


Positron -
new IDE from
Posit for
R/Python etc

The slides will be shared after!

Recap: using R projects and R env

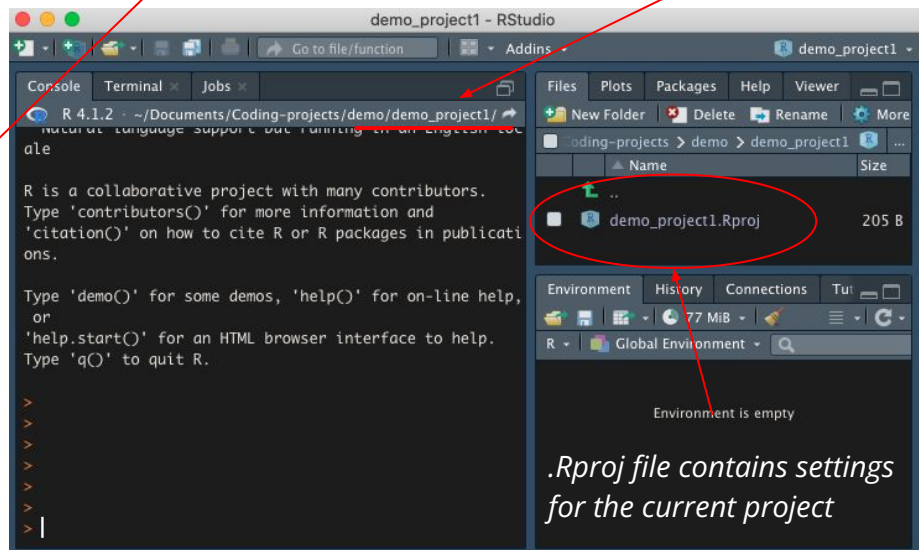
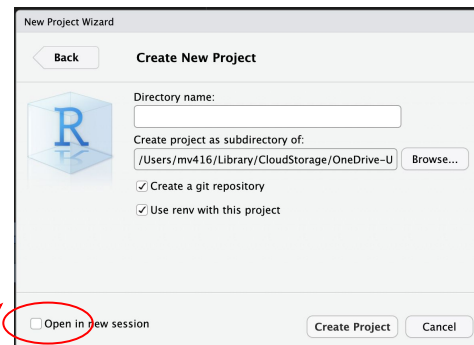
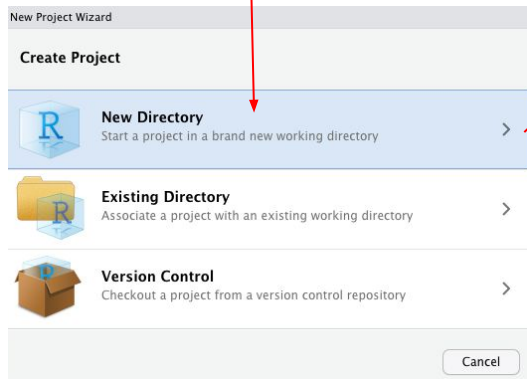
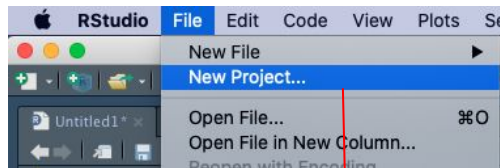
R projects



Reproducibility can be enhanced through intentionally organising projects with .Rproj, i.e. working in **project-oriented workflow**

Project-oriented workflow

Creating your new project in Rstudio as .Rproj:



Project directory

.Rproj file contains settings
for the current project

This means that you are essentially compartmentalizing your current project

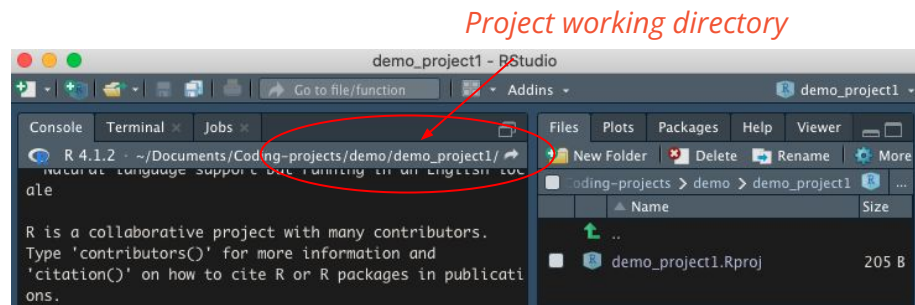
How working within .Rproj improves reproducibility:

File system discipline: project directory stores all your data, scripts, figures

File path discipline: The working directory is set to the project directory (e.g.

demo_project1/), so you don't need to *setwd()* or specify full paths to data (only internal subfolders that are relative to top directory)

Working directory intentionality: when working on project A, your working directory is set to project A's folder



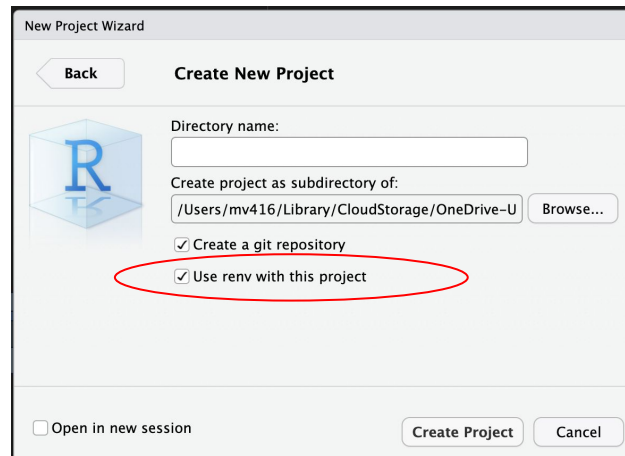
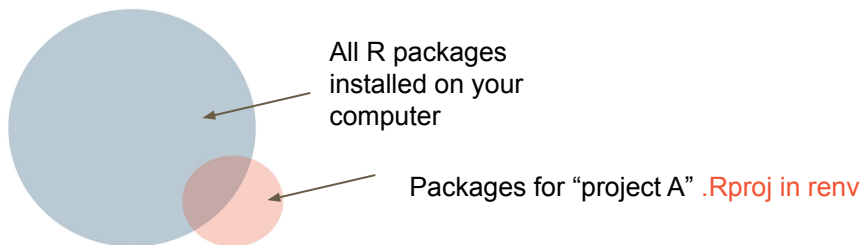
- The project **creates everything it needs**, within its workspace/folder, and **touches nothing it did not create**
- Any scripts are written assuming they will be **run from a fresh R session** within the project
- The project folder **can be moved anywhere**, and everything will still work (no paths will be broken)

renv for managing package versions within projects

renv package helps you create reproducible environments for your R projects



- Keeps track of all your installed packages
- Installs them in a new location (i.e. easy to share)
- Allows to come back to an old project without code breaking due to updates in package versions



Setting up your renv

```
install.packages("renv")
```

`renv::init()` – initialise your project env

This creates:

NB this is the same as

☒ Use renv with this project

- **renv/**: stores packages for the project.
- **renv.lock**: records packages and the exact versions used
- **.Rprofile**: ensures renv activates when the project opens

☐		.Rprofile	26 B
☐		renv	
☐		renv.lock	402 B
☐		test_project1.Rproj	205 B

When renv is initialised, your project gets its own project library - a private folder (**renv/library/**) where packages are installed just for this project.

This means package versions are isolated and recorded for each project, so updating a package in one project won't break others.

This makes collaboration and reproducibility much easier.

renv::snapshot() – update your env with newly installed packages (saves to renv.lock)

renv::restore() – when project is re-opened, load all your required packages

Using Git to track changes in your R projects

<https://happygitwithr.com/>

Checklist before we start

- GitHub account
- Personal access token
- Git is available on your laptop
- SSH-key (linking Git on your laptop with your GitHub account)

Git brief intro



Git - software on your computer that records your project history

Repository - a folder on your computer that Git is watching

GitHub - a website that hosts your repository online for sharing, back-up, and collaboration

Why use it?

- Improves reproducibility (e.g. keeps record of changes)
- Enables code sharing with colleagues / as a part of publication
- Can facilitate data/code integrity
- Important skill for anyone working with data (e.g. can showcase your work during job applications)

Why might you want to use git?

Your R project
on local PC

Local repository

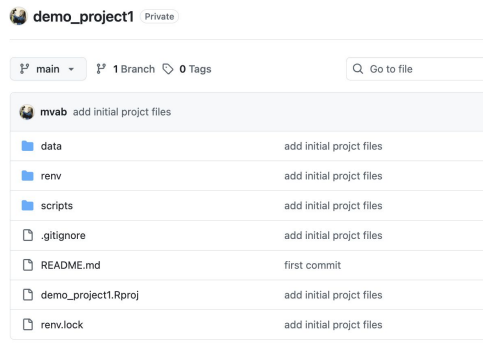
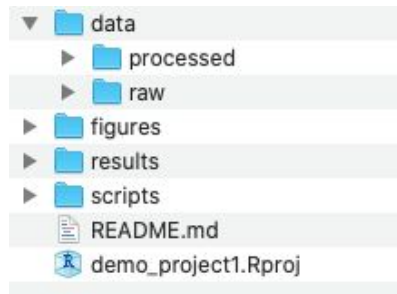


Your R project
as GitHub repo

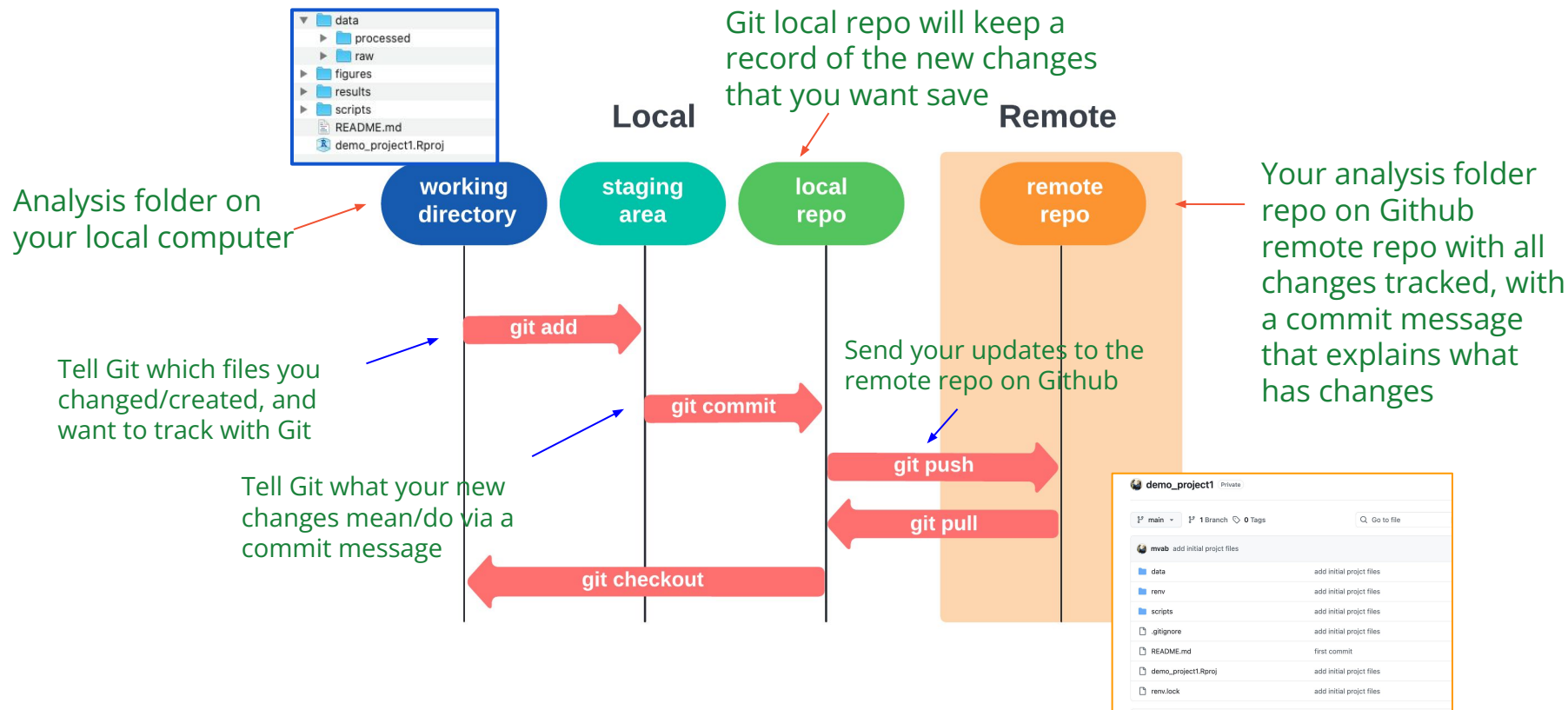
Remote repository



- Using git helps you keep track of your daily changes
- All your work is backed-up (as long as you do it regularly and intentionally)
- “Collaborating with yourself” in one place



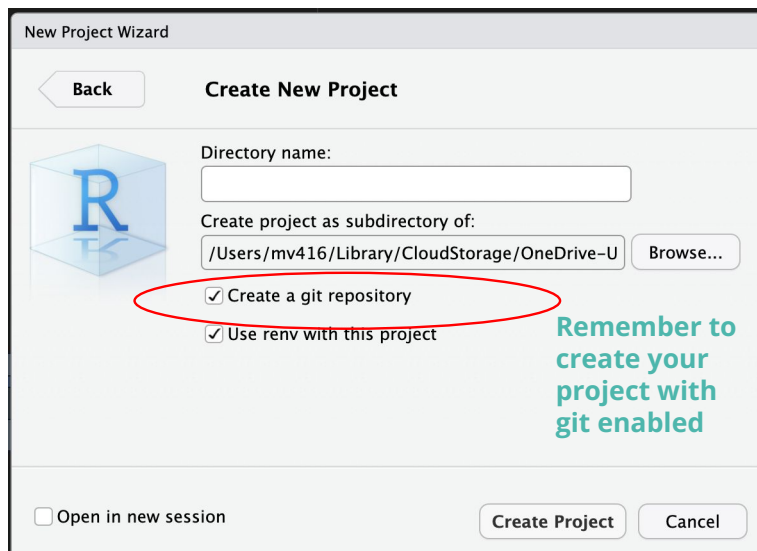
Basic git commands / actions



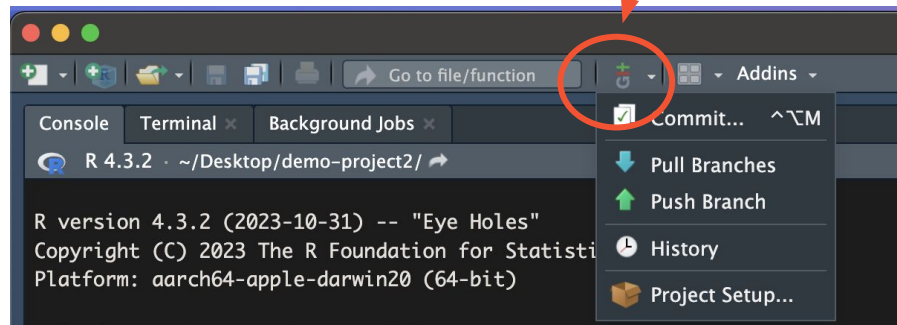
Using git with your .Rproj

Follow along if you can

- Rstudio user interface makes it easy to get started with git
- (but also can use the terminal to run git commands)



This means Git is enabled in the current project (use drop-down to interact with it)



-> Next need to link your local project folder to a GitHub repo

Create remote repo

After logging in your Github account:



To create a new repository:

Repositories -> New

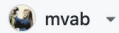
Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner *



Repository name *

test_project1

test_project1 is available.

Give repo the same name as your .Rproj folder

Great repository names are short and memorable. How about [legendary-memory](#)?

Description

0 / 350 characters

Make it private if it's ongoing work

2 Configuration

Don't change any configurations to keep it simple (except visibility)

Choose visibility *

Choose who can see and commit to this repository

Public

Start with a template

Templates pre-configure your repository with files.

No template

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

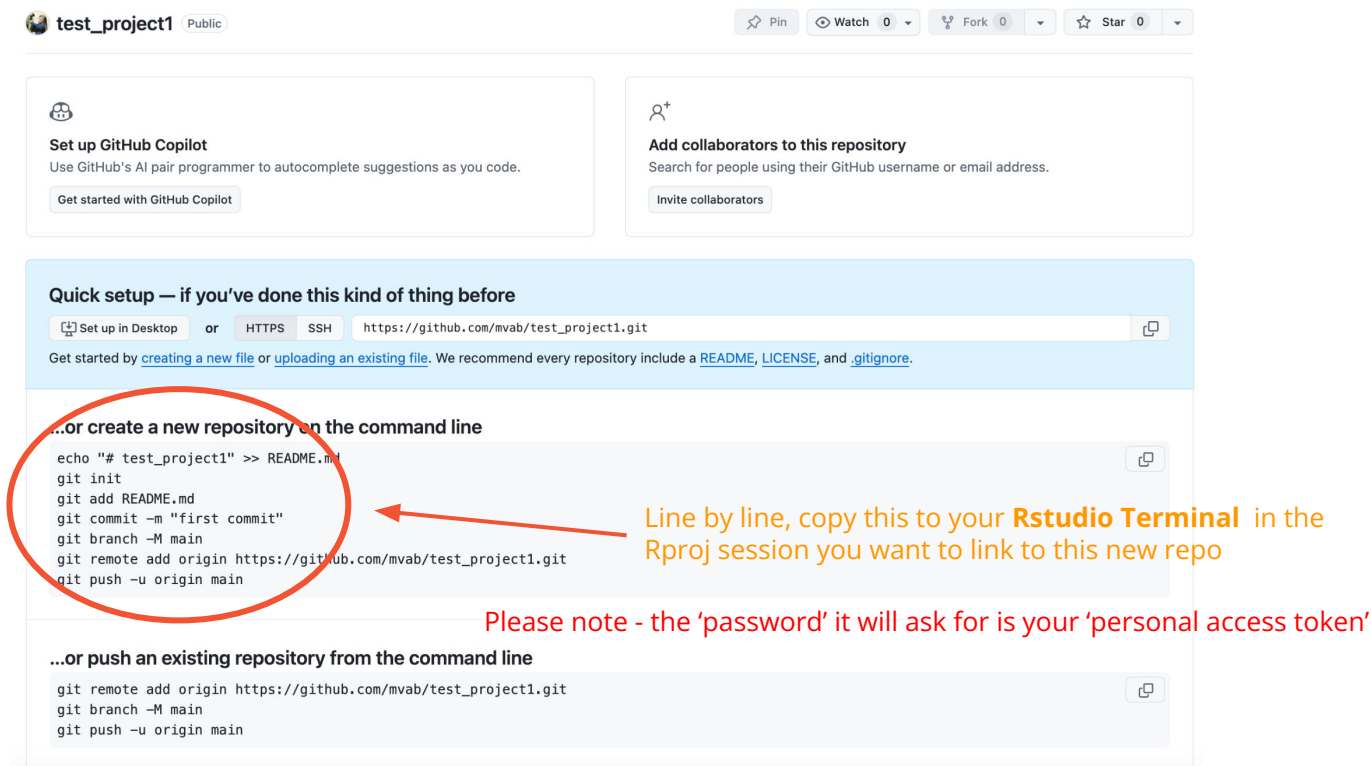
Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

GitHub makes it easy to link your .Rproj to the new repo:



The screenshot shows the GitHub interface for a repository named 'test_project1'. At the top, there are buttons for 'Pin', 'Watch' (0), 'Fork' (0), and 'Star' (0). Below these are two cards: 'Set up GitHub Copilot' and 'Add collaborators to this repository'. The main content area is titled 'Quick setup — if you've done this kind of thing before' and provides instructions for setting up the repository. A red circle highlights the command-line setup instructions, and a red arrow points from the text 'Line by line, copy this to your Rstudio Terminal in the Rproj session you want to link to this new repo' to the highlighted code. Another red arrow points from the text 'Please note - the 'password' it will ask for is your 'personal access token'' to the same code block.

test_project1 Public

Set up GitHub Copilot

Use GitHub's AI pair programmer to autocomplete suggestions as you code.

Get started with GitHub Copilot

Add collaborators to this repository

Search for people using their GitHub username or email address.

Invite collaborators

Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH `https://github.com/mvab/test_project1.git`

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# test_project1" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/mvab/test_project1.git
git push -u origin main
```

Line by line, copy this to your **Rstudio Terminal** in the Rproj session you want to link to this new repo

Please note - the 'password' it will ask for is your 'personal access token'

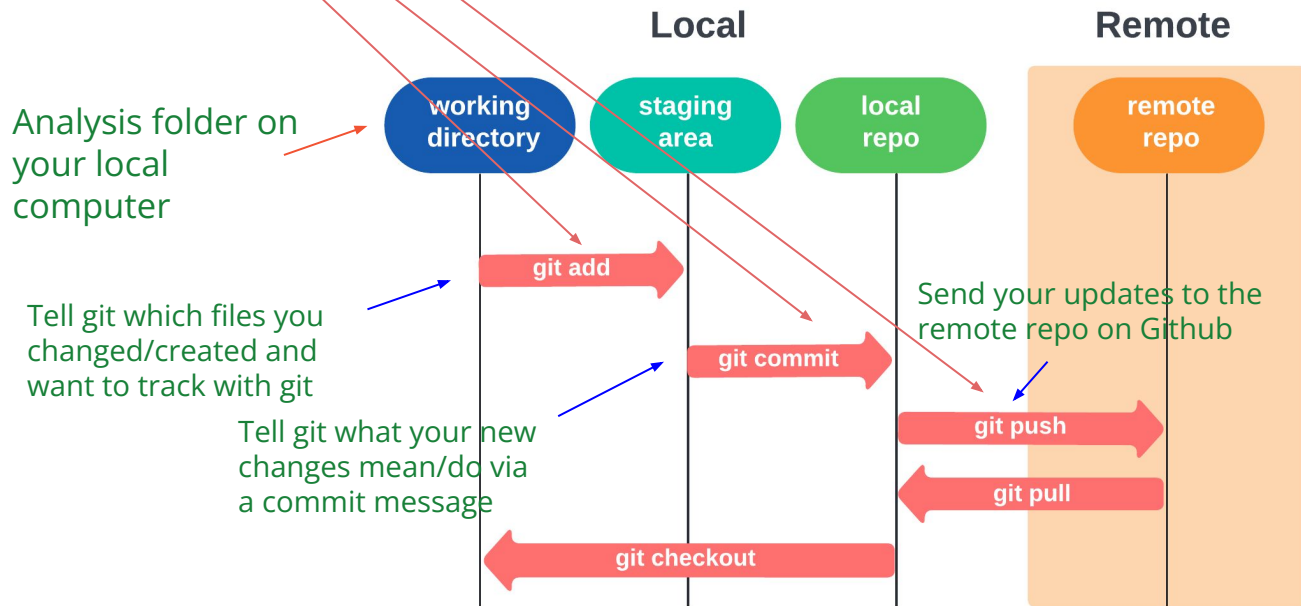
...or push an existing repository from the command line

```
git remote add origin https://github.com/mvab/test_project1.git
git branch -M main
git push -u origin main
```

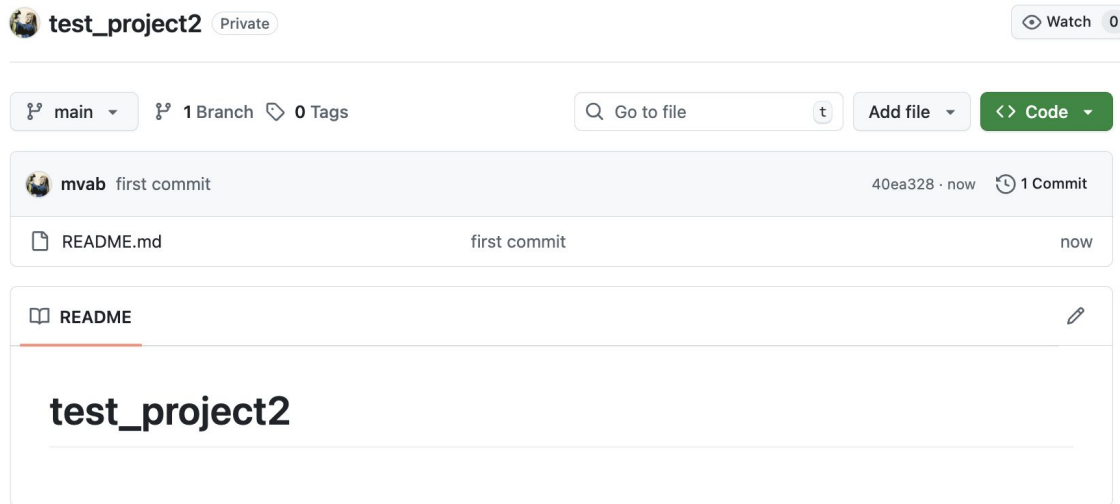
Only need
to do this
once!


```
echo "# test_project1" >> README.md  <- creating README.md file with "." text in it - can change it!
git init  <- initialises your project repo
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/mvab/test_project1.git
git push -u origin main
```

} initialises your project repo



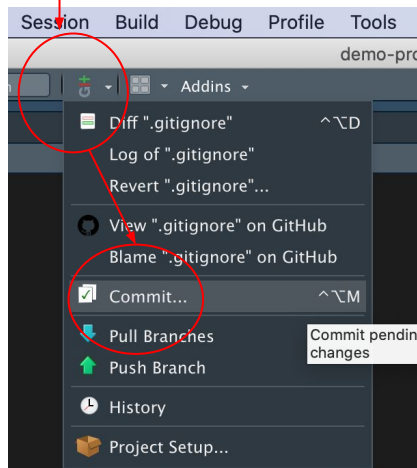
Remote repo after first commit



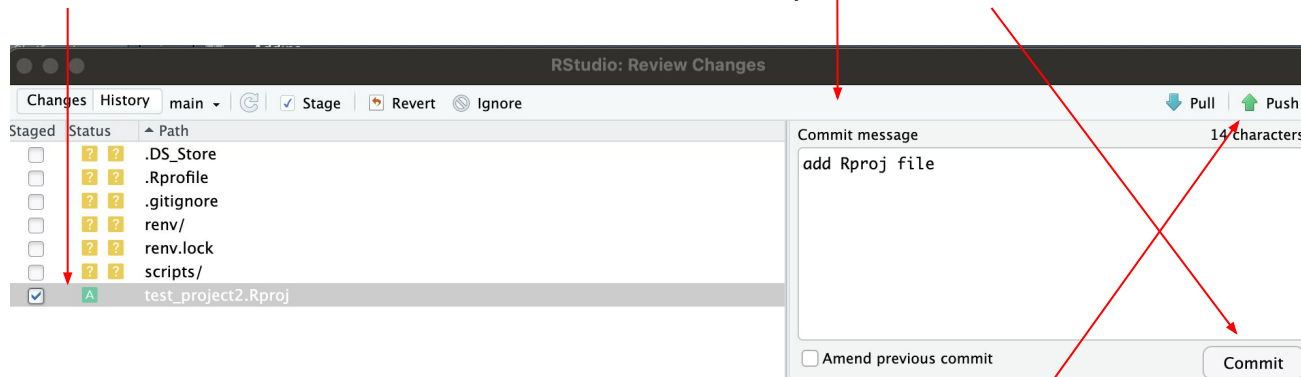
Only
README file
has been
added

Time to add other files!

- 1) Git button -> Commit



- 2) Select the files you want Git to **add** (track):



- 3) write your 'commit message', ie description of your change (tip: write it as an instruction), then press "**Commit**"

- 4) Press "**Push**" to send the change to remote repo

Your change has been added:

The screenshot shows a GitHub repository interface for 'test_project2', which is marked as 'Private'. At the top, there are navigation elements: a branch selector set to 'main', a branch count of '1 Branch', and a tag count of '0 Tags'. To the right is a search bar labeled 'Go to file' and buttons for 'Add file' and '<> Code'. Below this, a commit history table is displayed. The most recent commit is by user 'mvab' with the message 'add Rproj file', commit hash '85cc036', and timestamp '18 minutes ago'. It shows '2 Commits'. The commit details list two files: 'README.md' (first commit, 37 minutes ago) and 'test_project2.Rproj' (add Rproj file, 18 minutes ago). Red arrows point from green annotations to these entries. Below the commit history, the 'README' file is shown, containing the text 'test_project2'.

test_project2 Private Watch 0

main 1 Branch 0 Tags Go to file Add file <> Code

mvab add Rproj file 85cc036 · 18 minutes ago 2 Commits

README.md	first commit	37 minutes ago
test_project2.Rproj	add Rproj file	18 minutes ago

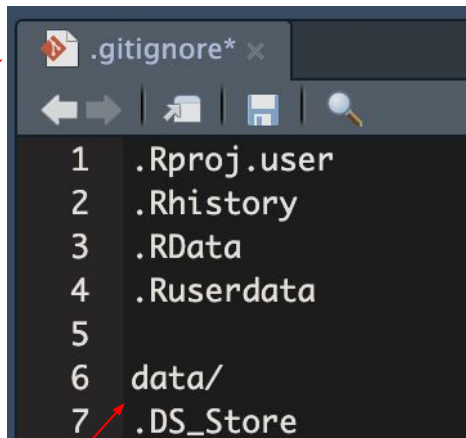
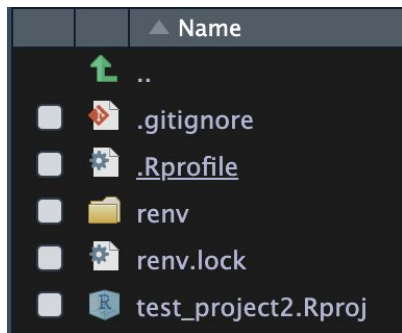
README

test_project2

File added

Commit message, ie description of what's new

.gitignore file - list files you don't want to store in remote repo



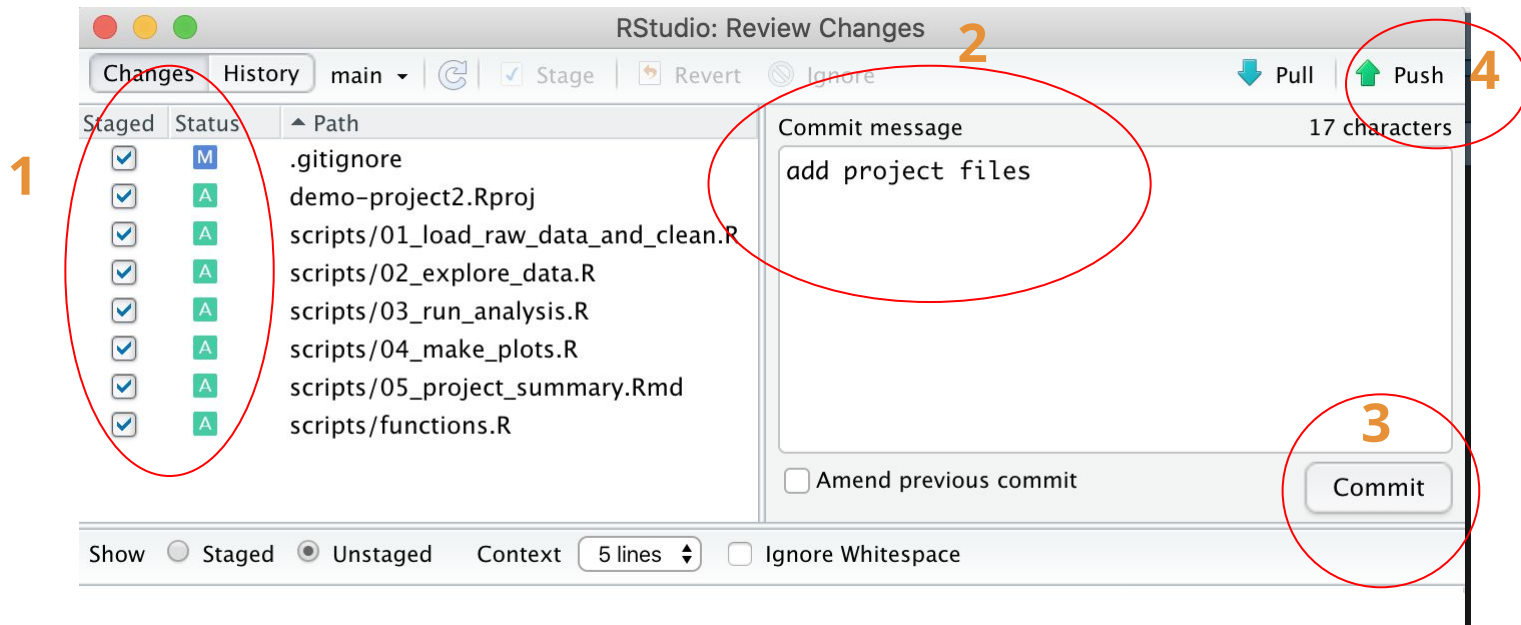
Add **data/** folder to *.gitignore* file so that your data files (if large or sensitive) are not committed to your project repo on Github

- list files and folders in your project folder in the *.gitignore* file - they will be ignore by Git (this prevents them from being accidentally added to GitHub repo)
- save and '**add**' file
- **commit** and **push** the updated *.gitignore* file to your repo

Now add more files:

Commit changes:

Add message:



Get files: <https://tinyurl.com/4czpk3cb>



test_project2

Private

Watch 0

main

1 Branch 0 Tags

Go to file

t

Add file

<> Code



mvab add all scripts

6da936d · now

5 Commits



renv

add renv files

now



scripts

add all scripts

now



.Rprofile

add renv files

now



.gitignore

add gitignore

1 minute ago



README.md

first commit

3 days ago



renv.lock

add renv files

now



test_project2.Rproj

add Rproj file

3 days ago



README



test_project2

Practise adding a specific change to a script:

1

```
1 library(readr)
2
3 # Read data ----
4 data_raw <- read_tsv("data/raw/raw_data_file.tsv")
5
6 # Clean data ----
7
8 ## step 1 ----
9
10 c = 1 + 2 # adding this change
11
12
13 ## step 2 ----
14
15 output <- my_useful_function(input)
16
17 ### step 2a ----
```

2

3

4

You will see what's changed here

Write a message that describes your change, then commit and push

```
RStudio: Review Changes

Changes History main Stage Revert Ignore Pull Push

Staged Status Path
[M] scripts/01_load_raw_data_and_clean.R

Commit message 12 characters
calculated c
[ ] Amend previous commit Commit

Show Staged Unstaged Context 5 lines Ignore Whitespace Unstage All

@@ -5,10 +5,13 @@ data_raw <- read_tsv("data/raw/raw_data_file.tsv")
5 5
6 6 # Clean data ----
7 7
8 8 ## step 1 ----
9 9
10 10 c = 1 + 2 # adding this change
11 11
12 12
10 13 ## step 2 ----
11 14
12 15 output <- my_useful_function(input)
13 16
14 17 ### step 2a ----
```


main ▾

demo-project2 / scripts /



mvab calculated c

..



01_load_raw_data_and_clean.R

calculated c



02_explore_data.R

add project files



03_run_analysis.R

add project files



04_make_plots.R

add project files



05_project_summary.Rmd

add project files



functions.R

add project files

Good practice principles

1. **Add** and **commit** your changes often and in small chunks
 - E.g. separate commits for different files and sections in your analysis steps
2. Write descriptive commit messages of what you've changed

Good

- Remove duplicate patient encounters from EHR extract
- Handle missing values using median imputation
- Engineer Comorbidity Index features
- Train XGBoost model for sepsis prediction

Bad

- fix
- update
- WIP
- final

3. Use .gitignore
4. Keep a project README
 - describes what your scripts do, and where external data is stored

Summary: using .Rproj for organising work

- “Reproducible project in R” means:
 - **File system discipline:** all files related to a single project are stored in a designated folder;
 - **Working directory discipline:** intentionally work in project directory when opening Rproj
 - **File path discipline:** paths are relative to the project directory (not hard-coded full paths!)
 - **Daily work habit:** Restarting R often and re-running your script under development from the top will help you catch issues early on
 - **Using git for version control**
- Practising these habits together will give you the biggest pay-off
 - Reproducing your analyses will be easy
 - Organising your projects will help you make sense of them in 6/12/etc months
 - Can move your project anywhere or share it with anyone without changing paths

Recommended and used resources

<https://www.tidyverse.org/blog/2017/12/workflow-vs-script/>

<https://richpauloo.github.io/2018-10-17-How-to-keep-your-R-projects-organized/>

<https://www.rforecology.com/post/organizing-your-r-studio-projects/>

<https://kkulma.github.io/2018-03-18-Prime-Hints-for-Running-a-data-project-in-R/>

<https://rstats.wtf/project-oriented-workflow.html>

<https://appsilon.com/rstudio-shortcuts-and-tips/>

<https://datacornering.com/my-favorite-rstudio-tips-and-tricks/>

<https://happygitwithr.com/>

<https://rstudio.github.io/renv/articles/renv.html>

<https://rstats.wtf/>

<https://youtu.be/fTVp7K38jgQ?si=ILLFHMV3t3qKT7sa>

Thank you!



University
of Exeter



Promoting Open and Reproducible Research at Exeter

University of Exeter UK Reproducibility Network (UKRN)

The University of Exeter is a member of [UK Reproducibility Network](https://www.ukrn.ac.uk/) (UKRN).

UKRN functions as a nationwide consortium driven by peers, with the overarching goal of maintaining the UK's position as a hub for cutting-edge research. This objective is achieved through an exploration of the elements that bolster resilient research, the facilitation of training initiatives, and the widespread sharing of exemplary methodologies. Through collaborative partnerships with external stakeholders, the UKRN ensures seamless alignment of endeavors within the sector.

**JOIN THE
UKRN EXETER
MAILING LIST**



Stay informed
on upcoming
event and
initiatives



<https://www.exeter.ac.uk/research/openresearch/reproducibility/ukrn/>